

DMA传输

DMA传输

一、学习目标

DMA原理

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. ADC参数配置

3. DMA参数配置

4. 写入程序

5. 编译

四、程序分析

五、实验现象

一、学习目标

1.了解DMA基本知识。

2.学习DMA的基本使用，将传输的ADC数据显示在串口助手上。

DMA原理

DMA（Direct Memory Access，直接存储器访问）是一种用于计算机系统的技术，使外设可以直接访问内存，而无需通过CPU来完成数据的传输。它使得数据在外设（如ADC、DAC、存储器等）和内存之间的交换能够高效地进行，减少了CPU的负担，提高了系统的整体性能。

原理：

- **触发数据传输：**DMA操作通常由外部设备（如ADC、传感器）或内部事件（如定时器溢出）触发。
- **初始化DMA控制器：**CPU向DMA控制器配置源地址、目标地址、传输数据大小、传输方向等参数。
- **直接传输数据：**一旦DMA控制器被激活，它能够直接从源地址（外设的输出缓冲区或内存）将数据传输到目标地址（内存或外设），而无需CPU干预。
- **传输完成后通知CPU：**DMA完成数据传输后，可以向CPU发送中断请求，告诉CPU数据已经准备好，或者传输已成功完成。

二、硬件搭建

MSPM0G3507的DMA控制器具有以下特点：

- 7个独立的传输通道；
- 可以配置的DMA通道优先级；
- 支持8位(byte)，16位(short word)、32位(word)和64位(long word)或者混合大小(byte 和 word)传输；
- 支持最大可达64K任意数据类型的数据块传输；

- 可配置的DMA传输触发源；
- 6种灵活的寻址模式；
- 单次或者块传输模式： 它共有7个DMA通道，各个通道可以独立配置，多种多样的数据传输模式可以适应不同应用场景的数据传输需要。

通过查看TI的数据手册，DMA功能除了常见的内存与外设间的地址寻址方式，还提供了Fill Mode和Table Mode两种拓展模式，DMA通道分为基本类型和全功能类型两种。

Table 4-1. Feature Comparison of Basic and Full-Feature DMA Channels

DMA Feature	Full-Feature Channel	Basic Channel
Repeated mode	✓	–
Early IRQ notification	✓	–
Block burst mode	✓	✓
Stride mode	✓	✓
Internal channel as trigger source	✓	✓
Extended Mode (Table and Fill Mode)	✓	–

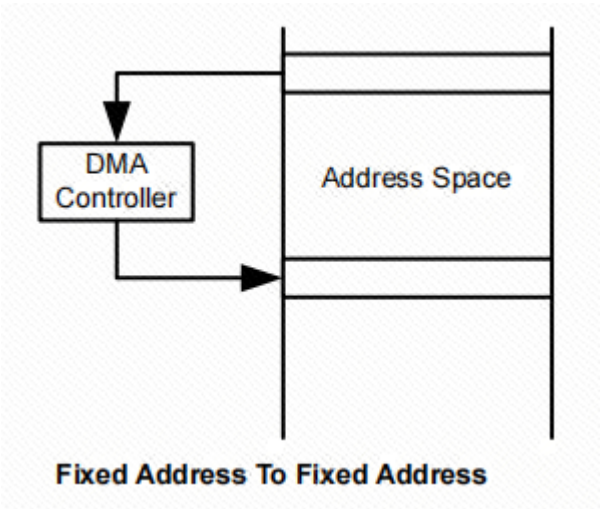
通过查看数据手册，只有全功能类型的DMA通道支持重复传输、提前中断以及拓展模式，基本功能的DMA通道支持基本的数据传输和中断，但是足够满足简单的数据传输要求。

MSPM0G3507的DMA支持四种传输模式，每个通道可单独配置其传输模式。例如，通道 0 可配置为单字或单字节传输模式，而通道 1 配置为块传输模式，通道 2 采用重复块传输模式。传输模式独立于寻址模式进行配置。任何寻址模式都可以与任何传输模式一同使用。可以传输三种类型的数据，可以通过 .srcWidth 和 .destWidth 控制位进行选择。源位置和目标位置可以是字节、短字或字数据。也支持以字节到字节、短字到短字、字到字或任何组合的方式进行传输。如下：

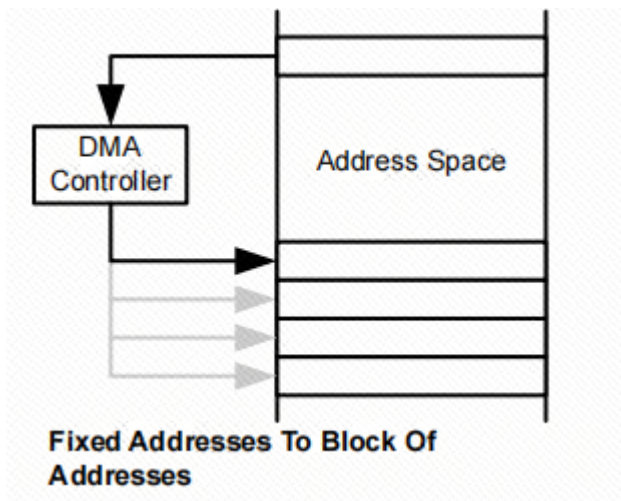
- 单字或单字节传输：每次传输都需要一个单独的触发。当 DMAxSZ 传输已经生成时 DMA 会被自动关闭。
- 块传输：一个整块将会在一个触发后传输。在块传输结束时 DMA 会被自动关闭。
- 重复单字或单字节传输：每次传输都需要一个单独的触发，DMA保持启用状态。
- 重复块传输：一个整块将会在一个触发后传输，DMA保持启用状态。

基于这三种模式，MSPM0G3507提供了6种寻址方式，

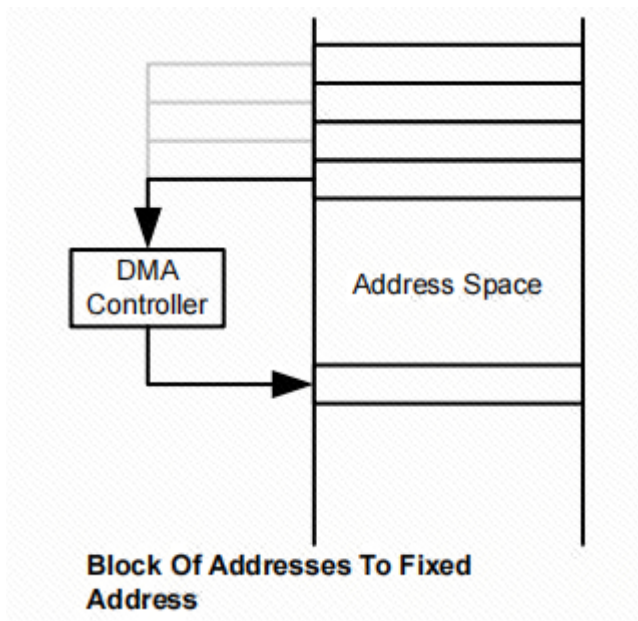
固定地址到固定地址：



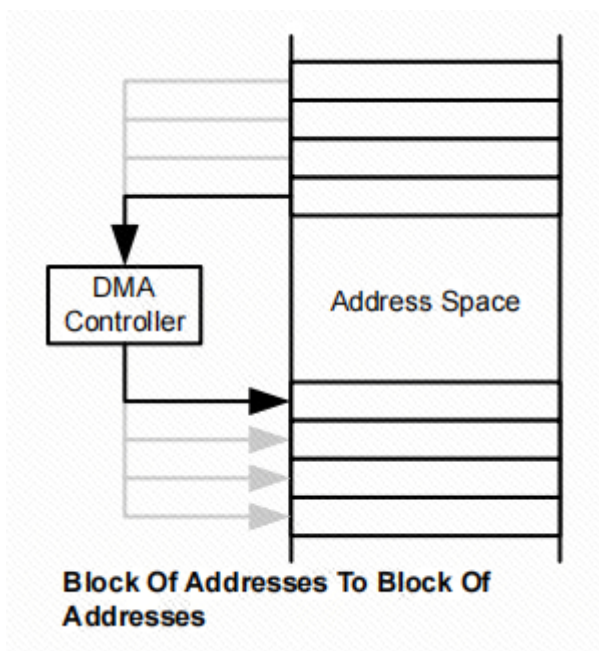
固定地址到地址块：



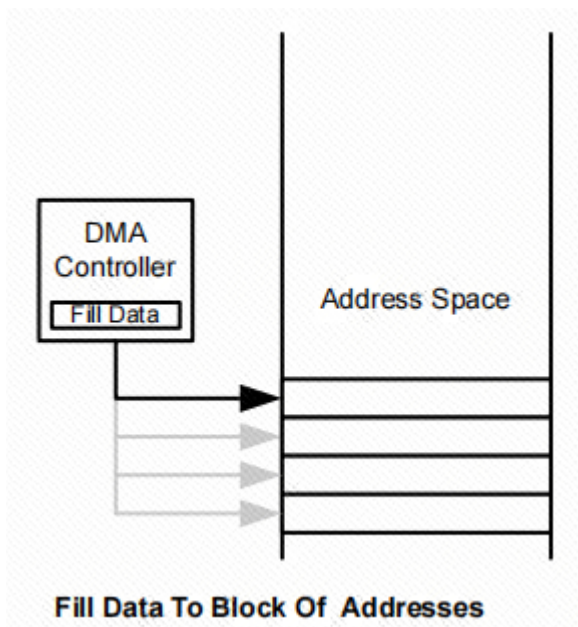
地址块到固定地址：



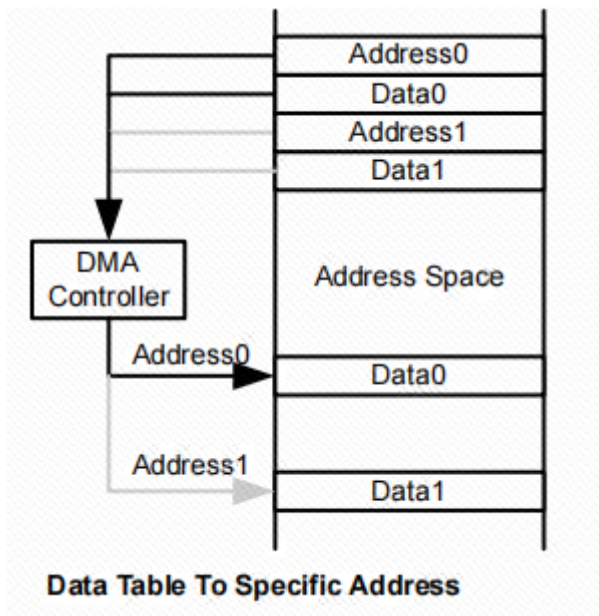
地址块到地址块：



将数据填充到地址块：



数据表到指定地址：

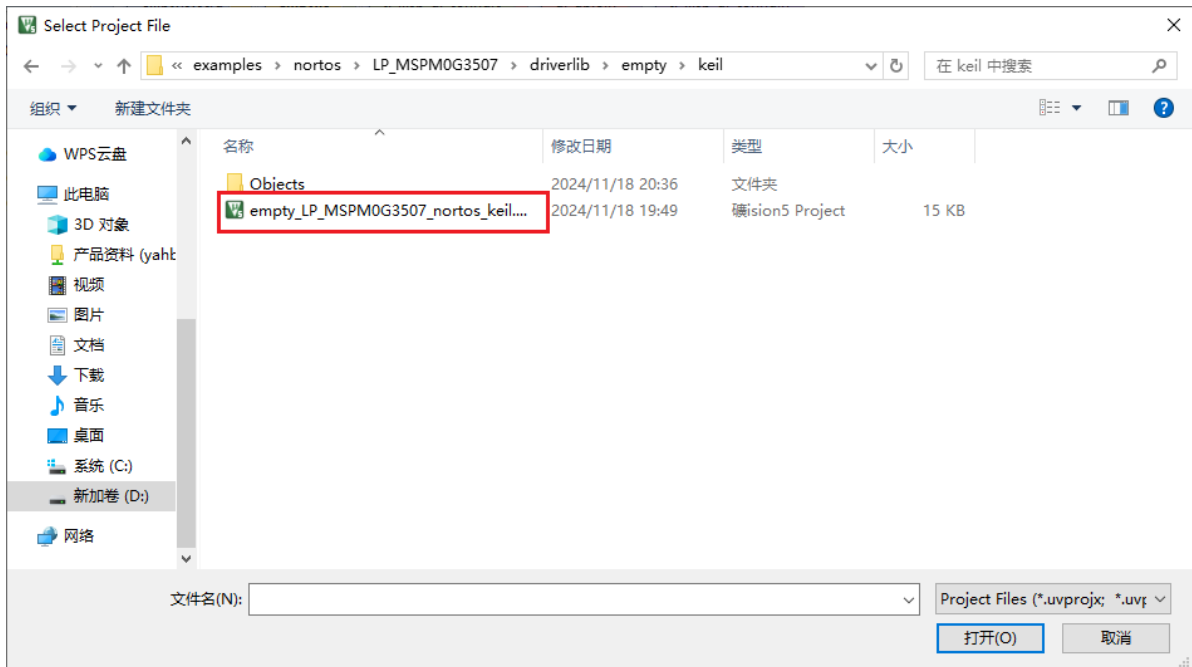


三、实验步骤

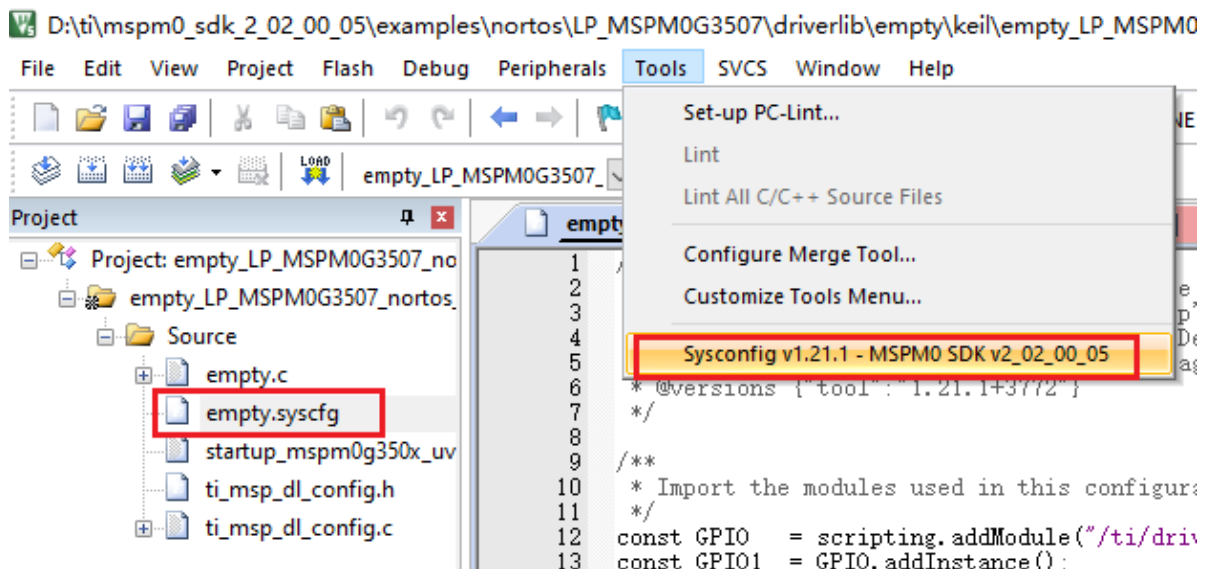
本案例将使用ADC采集PA27引脚的电压，通过DMA将ADC采集的数据直接搬运到指定地址中。

1. 打开SYSCONFIG配置工具

在KEIL中打开SDK中的空白工程 empty。



选中之后打开，在keil界面打开 empty.syscfg 文件，在 **empty.syscfg** 文件打开情况下，再在 Tools 栏选择打开 SYSCONFIG 的GUI界面。



2. ADC参数配置

这里ADC的配置与**ADC采集案例**相同。

配置ADC的分辨率和中断，本案例不使用中断，这里什么都不选。



3. DMA参数配置

DMA Configuration

Configure DMA Trigger
☒

Enable DMA Trigger
☒

DMA Samples Count
1

Enable DMA Triggers
MEM0 result loaded interrupt

DMA Channel

Name
DMA_CH0

Channel ID
0

Currently using a Full Channel.

Address Mode
Fixed addr. to Block addr.

Source Length
Half Word (2 Bytes)

Destination Length
Half Word (2 Bytes)

Destination Address Direction
Increment

Configure Transfer Size
☒

Transfer Size
10

Transfer Mode
Repeat Single

Source Address Increment
Do not change address after each transfer

Destination Address Increment
Do not change address after each transfer

Interrupt Configuration

Enable Channel Interrupt
☐

Enable Early Channel Interrupt
☐

参数说明：

Configure DMA Trigger： 是否开启DMA配置功能，这里选择开启。

Enable DMA Trigger： 使能DMA触发器。

DMA Samples Count： DMA样本计数。设置为1。

Enable DMA Triggers： DMA的触发方式。配置为 `MEM0 result loaded interrupt` ADC的结果寄存器0被装载中断。

Address Mode： DMA寻址模式。配置为 `Fixed addr. to Block addr.` 固定地址到数据块地址。

Source Length： 源地址的数据位宽。配置为 `Half word (2 Bytes)` 数据位宽2字节。

Destination Length： 目的地址的数据位宽配置。同上。

Destination Address Direction： 目的地址方向。选择 `Increment`，数据块地址开启增量。

Configure Transfer size： 是否配置传输数据量。这里选择开启。

Transfer Size： 传输的数据量。配置为10，一次数据块的传输数据量为10个数。

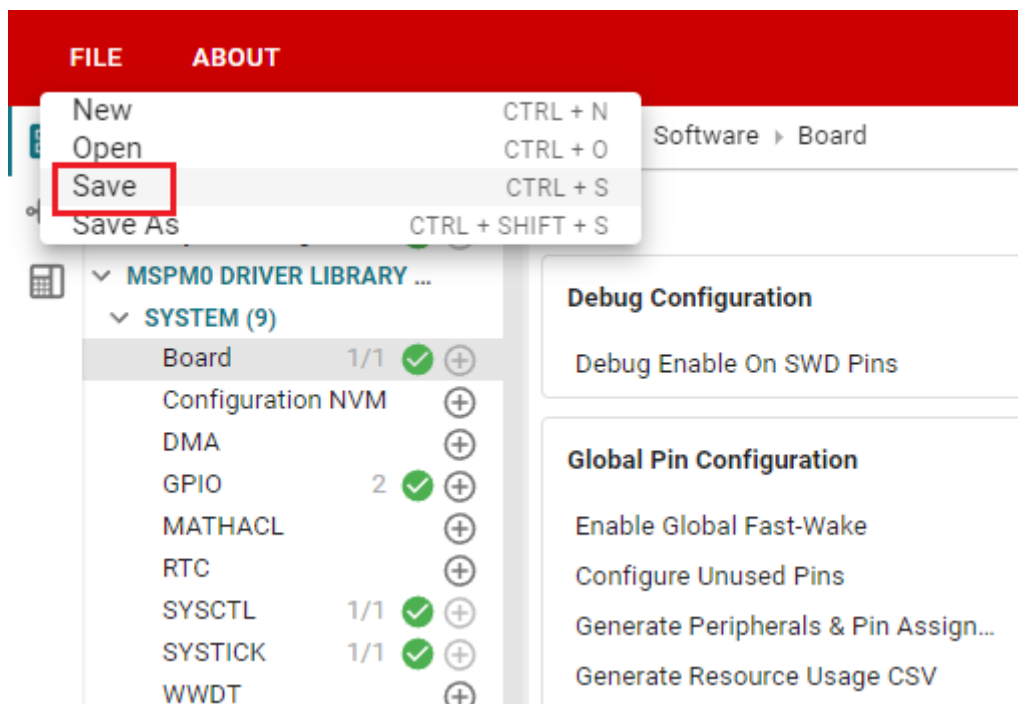
Transfer Mode： DMA传输模式。配置为 `Repeat Single` 重复单次传输。

Source Address Increment： 源地址是否增量。不开启。

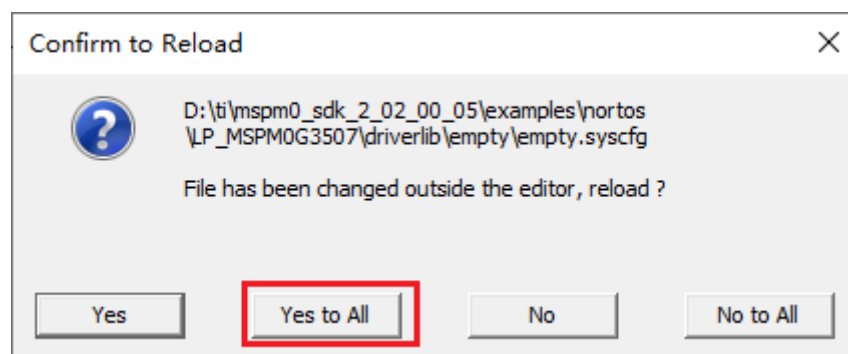
Destination Address Increment： 目的地址是否增量。不开启。

Enable Channel Interrupt： 是否开启DMA通道中断。不开启。

点击 `SAVE` 将 `SYSCONFIG` 中的配置保存，之后关掉 `SYSCONFIG` 回到keil。



在弹窗中选择 Yes to All



同样我们还需要确认ti_msp_dl_config.c和ti_msp_dl_config.h文件是否更新。直接编译，编译就会自动更新到keil中。如果没有更新，我们也可以把 SYSCONFIG 中的文件内容复制过来。

4. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"
#include "stdio.h"

volatile unsigned int delay_times = 0;
volatile bool gCheckADC;          //ADC采集成功标志位  ADC acquisition success flag
volatile uint16_t ADC_VALUE[20];

void delay_ms(unsigned int ms); //搭配滴答定时器的精准毫秒级延时  Precise millisecond
delay with tick timer
unsigned int adc_getValue(unsigned int number); //读取ADC的数据  Read ADC data

/*****串口重定向 Serial port
redirection*****/
#if !defined(__MICROLIB)
//不使用微库的话就需要添加下面的函数
// If you don't use the micro library, you need to add the following function
#if (__ARMCLIB_VERSION <= 6000000)
```



```

//如果编译器是AC5 就定义下面这个结构体
//If the compiler is AC5, define the following structure
struct __FILE
{
    int handle;
};
#endif

FILE __stdout;

//定义_sys_exit()以避免使用半主机模式
// Define _sys_exit() to avoid using semihosting mode
void _sys_exit(int x)
{
    x = x;
}
#endif
//printf函数重定义 printf function redefinition
int fputc(int ch, FILE *stream)
{
    //当串口0忙的时候等待，不忙的时候再发送传进来的字符
    // wait when serial port 0 is busy, and send the incoming characters
    when it is not busy
    while( DL_UART_isBusy(UART_0_INST) == true );

    DL_UART_Main_transmitData(UART_0_INST, ch);

    return ch;
}
/*****/

int main(void)
{
    unsigned int adc_value = 0;
    unsigned int voltage_value = 0;
    int i = 0;

    SYSCFG_DL_init();

    //清除串口中断标志 clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //开启串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);

    //设置DMA搬运的起始地址 Set the starting address of DMA transfer
    DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC0-
>ULLMEM.MEMRES[0]);
    //设置DMA搬运的目的地址 Set the destination address of DMA transfer
    DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC_VALUE[0]);
    //开启DMA Enable DMA
    DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
    //开启ADC转换 Start ADC conversion
    DL_ADC12_startConversion(ADC_VOLTAGE_INST);

    printf("adc_dma Demo start\r\n");

    while (1)
    {

```



```

        //获取ADC数据 Get ADC data
        adc_value = adc_getValue(10);
        printf("adc value:%d\r\n", adc_value);

        //将ADC采集的数据换算为电压 Convert the data collected by ADC into
voltage

        voltage_value = (int)((adc_value/4095.0*3.3)*100);

        printf("voltage value:%d.%d%d\r\n",
        voltage_value/100,
        voltage_value/10%10,
        voltage_value%10 );

        delay_ms(1000);
    }
}
//延时函数 Delay function
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

//读取ADC的数据 Read ADC data
unsigned int adc_getValue(unsigned int number)
{
    unsigned int gAdcResult = 0;
    unsigned char i = 0;

    //采集多次累加 Collect multiple times and accumulate
    for( i = 0; i < number; i++ )
    {
        gAdcResult += ADC_VALUE[i];
    }
    //均值滤波 Mean Filter
    gAdcResult /= number;

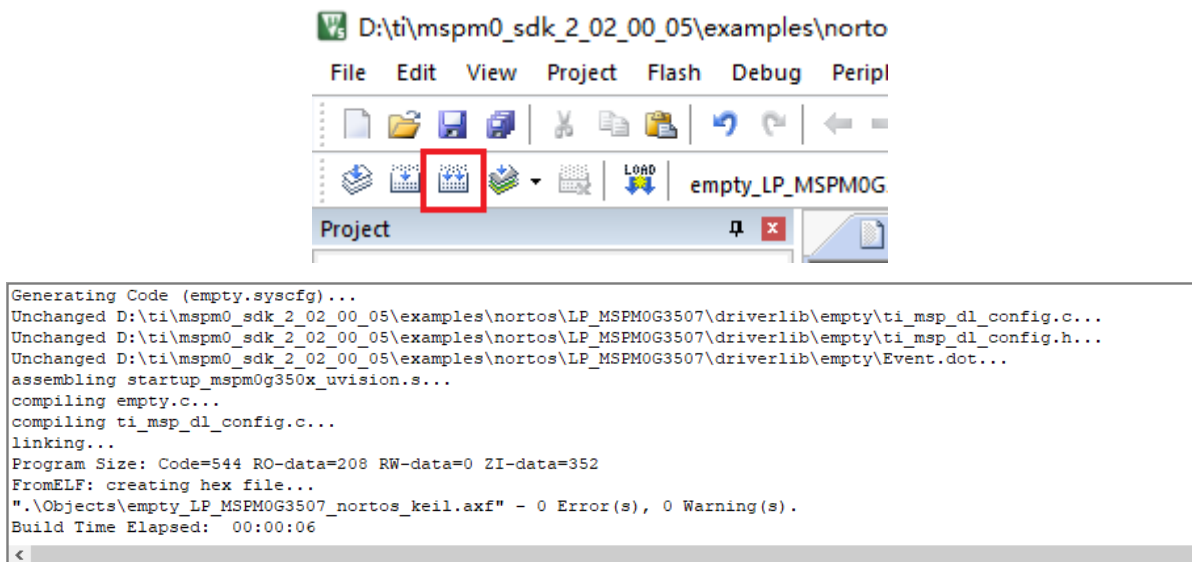
    return gAdcResult;
}

//滴答定时器的中断服务函数 Tick timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

```

5. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



四、程序分析

- empty.c

```
//读取ADC的数据 Read ADC data
unsigned int adc_getValue(unsigned int number)
{
    unsigned int gAdcResult = 0;
    unsigned char i = 0;

    //采集多次累加 collect multiple times and accumulate
    for( i = 0; i < number; i++ )
    {
        gAdcResult += ADC_VALUE[i];
    }
    //均值滤波 Mean Filter
    gAdcResult /= number;

    return gAdcResult;
}
```

该函数通过对 `ADC_VALUE` 数组中的数据进行累加并计算平均值来滤波。这样可以减少单次ADC采样的噪声。`number` 是采样次数，通常设置为10或更多，以提高数据的稳定性。

```
int main(void)
{
    unsigned int adc_value = 0;
    unsigned int voltage_value = 0;
    int i = 0;

    SYSCFG_DL_init();

    //清除串口中断标志 Clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //开启串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);

    //设置DMA搬运的起始地址 Set the starting address of DMA transfer
    DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC0-
    >ULLMEM.MEMRES[0]);
}
```

```

//设置DMA搬运的目的地址 Set the destination address of DMA transfer
DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC_VALUE[0]);
//开启DMA Enable DMA
DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
//开启ADC转换 Start ADC conversion
DL_ADC12_startConversion(ADC_VOLTAGE_INST);

printf("adc_dma Demo start\r\n");

while (1)
{
    //获取ADC数据 Get ADC data
    adc_value = adc_getValue(10);
    printf("adc value:%d\r\n", adc_value);

    //将ADC采集的数据换算为电压 Convert the data collected by ADC into
voltage

    voltage_value = (int)((adc_value/4095.0*3.3)*100);

    printf("voltage value:%d.%d%d\r\n",
    voltage_value/100,
    voltage_value/10%10,
    voltage_value%10 );

    delay_ms(1000);
}
}

```

调用 `SYSCFG_DL_init()` 函数初始化系统,清除并使能串口中断,用于串口数据传输。设置DMA源地址为 `ADC0->ULLMEM.MEMRES[0]`, 该地址存储ADC的转换结果。设置DMA目的地址为 `ADC_VALUE[0]`, 将DMA结果存储到该数组。启用DMA通道并开始ADC转换。通过 `adc_getValue(10)` 获取10次ADC采样结果并进行均值滤波, 返回一个稳定的ADC值。将ADC值转换为电压值, 假设ADC为12位分辨率(4095为最大值), 转换为0-3.3V范围的电压值。

五、实验现象

程序下载完成之后, 对串口助手进行如下图配置之后, 调节电位器旋钮, 向PA27引脚输出电压, ADC将数据采集装载到ADC结果寄存器0后, DMA被触发, DMA将会把数据搬运到内存变量ADC_VALUE中, 通过直接调用ADC_VALUE即可得到数据。

